Adaptive Traffic Signal Control using Tabular Reinforcement Learning: A Comparative Study of Q-Learning and SARSA

Student: Daksh Gupta | **Roll No.:** _____
Course: Reinforcement Learning, Final Year B.Tech (CSE), Manipal University Jaipur
Faculty: _____ | **Category:** Mini Project

Abstract

Fixed-time traffic signals ignore real-time demand, which causes avoidable queues and delay at urban intersections, especially during peak hours. This project formulates signal control at an isolated four-way intersection as a Markov Decision Process and trains two tabular temporal-difference agents, Q-Learning (off-policy) and SARSA (on-policy), to decide at each 5-second step whether to keep or switch the current green phase. A custom Python simulator models Poisson vehicle arrivals with a time-varying peak-hour profile, saturation-flow discharge, a yellow interval and a minimum-green safety constraint. The reward is the negative total queue length. Agents were trained for 200 one-hour episodes over five random seeds and evaluated on unseen traffic against random, fixed-time and greedy longest-queue controllers. Q-Learning reduced average waiting time per vehicle to 9.7 s, a 42% reduction over the fixed-time controller (16.8 s) and 20% over the greedy actuated controller (12.1 s), with 59 fewer phase changes per hour than the greedy controller. SARSA performed comparably. The results show that even compact tabular RL can learn demand-responsive signal timing.

1. Introduction

Background and motivation. Traffic congestion wastes fuel, increases emissions and lengthens commutes in Indian cities. Most signals still run pre-timed plans derived from historical counts [7], which cannot react when demand shifts within an hour. Reinforcement learning (RL) is attractive because a controller can learn a timing policy from interaction with traffic, without a hand-built model of the flow [4], [9].

Problem statement. Given the current queues and signal state at an intersection, choose when to switch the green phase so that the total vehicle delay is minimised, while respecting a minimum green time and the lost time of each yellow interval.

Objectives. (i) Model the intersection as an MDP and build a reproducible simulator; (ii) implement Q-Learning and SARSA from scratch; (iii) compare them with three baselines on waiting time, queue length, throughput and switching frequency; (iv) analyse convergence and limitations.

2. Methodology

Environment. One four-way intersection with lanes N, S, E and W and two phases (phase 0: North-South green; phase 1: East-West green). Each step represents 5 s, so an episode of 720 steps is one hour. Arrivals per lane are Poisson with base rates of 0.45 (N/S) and 0.25 (E/W) vehicles/step, multiplied by Gaussian peaks so that North-South demand rises by 60% around 40% of the hour and East-West demand doubles near the end (tidal flow). A green lane discharges up to 2 vehicles per step. Switching phase triggers one all-red/yellow step with no discharge, and a switch is only allowed after 2 steps (10 s) of green.

State space. The state is the tuple (NS-queue level, EW-queue level, phase, time-in-phase level). Directional queues are binned at {0, 1-2, 3-4, 5-7, 8-12, 13+} vehicles and time-in-phase at {0-1, 2-3, 4-7, 8+} steps, giving 6 x 6 x 2 x 4 = 288 discrete states. **Action space:** {0 = keep phase, 1 = switch phase}. **Reward:** r_t = -(sum of queued vehicles after step t), scaled by 1/10 during learning. By Little's law, minimising the time-integral of queue length minimises total delay.

Algorithms. Both agents keep a Q-table and act epsilon-greedily. Q-Learning [2] uses the off-policy target r + gamma $\max_{a'}$ Q(s',a'), while SARSA [3] uses the on-policy target r + gamma Q(s',a') with a' the action actually taken next. The update is Q(s,a) ← Q(s,a) + alpha[target - Q(s,a)] [1]. Ties are broken randomly.

Table 1. Hyperparameters used for both agents.

Hyperparameter	Value	Hyperparameter	Value
Learning rate alpha	0.1	Discount gamma	0.95
Initial epsilon	1.0	Final epsilon	0.02
Epsilon decay / episode	0.97	Training episodes	200
Training seeds	5	Evaluation episodes	20

Training strategy and baselines. Each agent is trained for 200 episodes on five seeds; learning curves report the mean and standard deviation across seeds. The greedy policy is then evaluated on 20 unseen episodes. Baselines: *Random* (switch with probability 0.5), *Fixed-Time* (switch every 6 steps, i.e. 30 s green, a Webster-style plan [7]) and *Longest-Queue* (an actuated rule that always serves the longer direction, related to max-pressure control [8]). Metrics are average waiting time per vehicle (queue-time integral divided by vehicles served), average queue length, throughput and number of phase switches.

Software framework and architecture. Python 3 with NumPy for the simulator and agents and Matplotlib for plots; no external RL library is used, so every update is transparent. Fig. 1 shows the agent-environment loop.

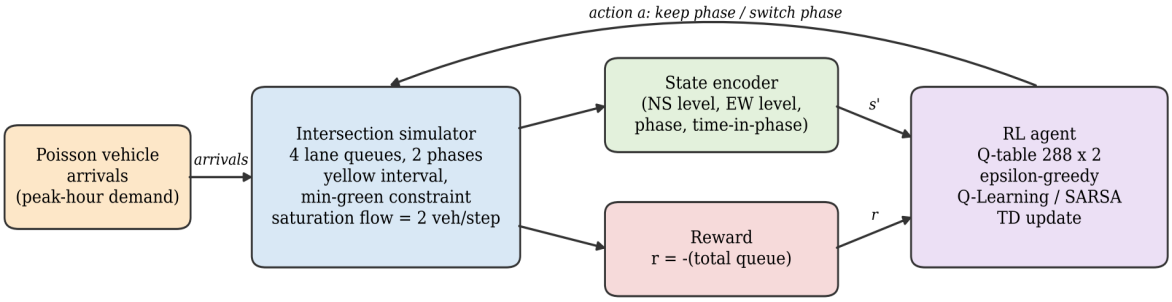


Fig. 1. System architecture and agent-environment interaction loop.

3. Experimental Results

Training behaviour. Fig. 2 shows that both agents improve rapidly during the first 50 episodes as epsilon decays and then stabilise. Waiting time during training falls below the fixed-time baseline after about 25 episodes and settles near 10 s. The narrow shaded bands show that convergence is consistent across seeds.

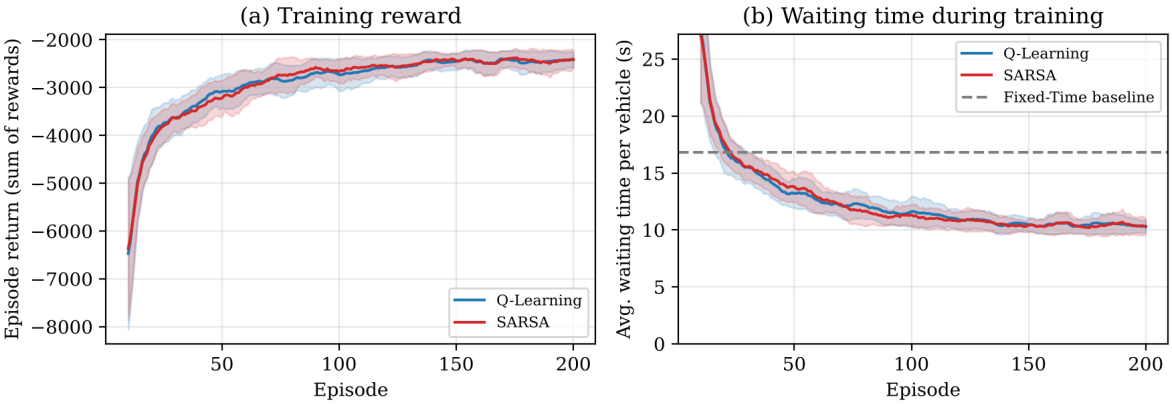


Fig. 2. Learning curves (mean +/- std over 5 seeds, 10-episode moving average): (a) episode return, (b) average waiting time per vehicle.

Table 2. Evaluation on 20 unseen episodes (mean ± std). Uniform = constant demand without peaks (generalisation test).

Controller	Avg. wait (s) peak	Avg. queue (veh)	Throughput (veh/h)	Switches /h	Avg. wait (s) uniform
Random	43.19 ± 17.82	13.85	1154	240	19.32
Fixed-Time	16.79 ± 1.56	5.39	1155	119	14.22
Longest-Queue	12.13 ± 0.64	3.90	1158	280	9.68
SARSA	9.92 ± 1.12	3.19	1158	229	8.62
Q-Learning	9.66 ± 0.28	3.11	1158	221	8.79

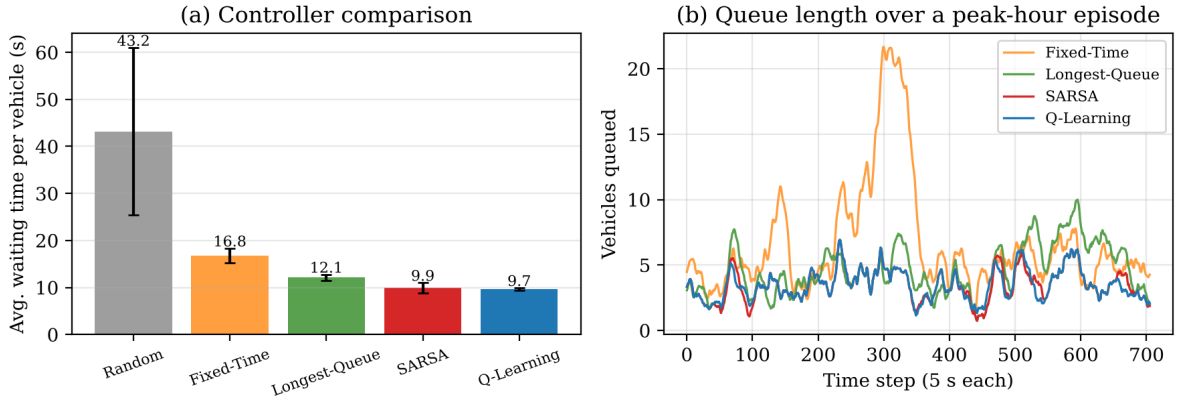


Fig. 3. (a) Average waiting time per controller under peak-hour demand; (b) total queue during one evaluation episode (15-step moving average).

Performance analysis. Table 2 and Fig. 3 show that both RL agents outperform all baselines. Q-Learning achieved 9.66 s average waiting time, 42.5% lower than Fixed-Time and 20.4% lower than Longest-Queue; SARSA reached 9.92 s. Throughput is nearly identical for all controllers because demand is below capacity, so the gain comes from serving vehicles sooner rather than serving more. Fig. 3(b) explains the gap: the fixed-time plan cannot react to the North-South surge around step 300 and its queue climbs above 20 vehicles, while the RL agents keep it near 5. The greedy controller reacts but switches too often (280 vs 221 switches/h), wasting a yellow interval each time; the RL agents learned to hold green slightly longer until the opposing queue justifies the lost time. On constant (uniform) demand, never seen in training, the agents still beat every baseline, indicating that the policy depends on queue state rather than memorising the peak profile. Q-Learning and SARSA converge to similar policies here because the minimum-green rule removes the risky exploratory actions where SARSA's on-policy caution usually matters; Q-Learning showed slightly lower variance across episodes.

4. Conclusion

Tabular Q-Learning and SARSA learned adaptive signal timing that cut average waiting time by about 42% compared with a fixed-time plan and about 20% compared with a greedy actuated rule, using a 288-state table that trains in about a minute on a laptop.

Limitations. The simulator is a single isolated intersection with two phases, no turning movements, pedestrians or vehicle dynamics. The hand-crafted queue bins limit resolution, and tabular methods do not scale to large networks or continuous sensor inputs. Results are from simulation and are not yet validated on real traffic data.

Future scope. (i) Replace the Q-table with a Deep Q-Network [5] or Double/Dueling DQN to use raw lane occupancy; (ii) move to the SUMO microscopic simulator with real Jaipur junction counts; (iii) multi-agent coordination across corridors for green waves [6], [9]; (iv) add pedestrian phases and emergency-vehicle priority to the reward.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3-4, pp. 279-292, 1992.

[3] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Cambridge Univ. Eng. Dept., Tech. Rep. CUED/F-INFENG/TR 166, 1994.

[4] B. Abdulhai, R. Pringle, and G. J. Karakoulas, "Reinforcement learning for true adaptive traffic signal control," *J. Transp. Eng.*, vol. 129, no. 3, pp. 278-285, 2003.

[5] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529-533, 2015.

[6] H. Wei, G. Zheng, H. Yao, and Z. Li, "IntelliLight: A reinforcement learning approach for intelligent traffic light control," in *Proc. ACM SIGKDD*, 2018, pp. 2496-2505.

[7] F. V. Webster, "Traffic signal settings," Road Research Laboratory, London, U.K., Tech. Paper 39, 1958.

[8] P. Varaiya, "Max pressure control of a network of signalized intersections," *Transp. Res. Part C*, vol. 36, pp. 177-195, 2013.

[9] H. Wei, G. Zheng, V. Gayah, and Z. Li, "A survey on traffic signal control methods," arXiv:1904.08117, 2019.